

Grim return to the planet of apes!!

the output should be

Grim return to the planet of apes!!

5. Can an array of pointers to strings be used to collect strings from the keyboard? If not, why not?
6. Write a program to sort a set of names stored in an array in alphabetical order.
7. Write a program to delete all vowels from a sentence. Assume that the sentence is not more than 80 characters long.
8. Write a program that takes a set of names of individuals and abbreviates the first, middle and other names except the last name by their first letter.



Further Readings

Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

YashvantKanetkar, Let us C



Web Links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com

Unit 12: Dynamic Memory Management

CONTENTS

Objectives

Introduction

12.1 Dynamic Memory Allocation

12.2 malloc() - memory allocation, size of, and free

12.3 calloc and realloc

12.4 free

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Dynamic Memory Management functions
- malloc() and calloc()
- realloc() and free()

Introduction

The Dynamic Memory Allocation concept is utilized to solve the array problem. An array is a set of values with a fixed number of elements. You can't adjust the size of an array once it's been declared. Programmers can use Dynamic Memory Allocation to allocate memory during runtime.

Because the static representation of a linear ordered list using an array wastes resources and, in some situations, causes overflows. We no longer want to pre-allocate memory to any linear list; instead, we want to allocate memory to elements as they are added to the list. This necessitates memory allocation that is dynamic.

12.1 Dynamic Memory Allocation

There are several limitations in static memory allocation:

This is done in RAM dedicated solely to a programme, which is frequently limited in capacity. The size of a static array is fixed. We won't be able to expand it to accommodate situations that require more elements. As a result, we'll likely to declare larger arrays than necessary, resulting in memory waste. We also can't lower array size to conserve memory when fewer array elements are necessary. Advanced data structures such as linked lists, trees, and graphs, which are needed in most real-world programming situations, are not possible (or efficient) to develop.

C has a feature called dynamic allocation that is quite unique (amongst high level languages). It allows us to design data types and structures of any size and duration to meet the needs of our programmes. Dynamic memory allocation occurs when memory is allocated at runtime, that is, when a programme is running. Pointers and four common library functions are used in dynamic memory management.

Programming language (C) provides several functions for memory allocation and management, namely, malloc, calloc, realloc and free. Functions are defined in the <stdlib.h> header file.

Static Vs Dynamic memory allocation

Static Memory Allocation	Dynamic Memory Allocation
variables get allocated permanently	variables get allocated only if your program unit gets active
Allocation is done before program execution	Allocation is done during program execution
It uses the data structure called stack for implementing static allocation	It uses the data structure called heap for implementing dynamic allocation
Less efficient	More efficient
There is no memory reusability	There is memory reusability and memory can be freed when not required
Memory is allocated at compile time.	memory is allocated at run time

12.2 malloc() - memory allocation, size of, and free

It's used to allocate a single block of memory with the specified size dynamically. It returns a void pointer that can be cast into any type of pointer. It does not perform memory initialization during execution. Initially, it has garbage value. If memory is insufficient, it returns NULL.

The Function malloc is most commonly used to attempt to “grab” a continuous portion of memory. It is defined by:

```
void *malloc(size_t number_of_bytes)
```

Syntax

```
ptr = (cast-type*) malloc(byte-size)
```

Example

```
ptr = (int*) malloc(20 * sizeof(int));
```

That is to say it returns a pointer of type void * that is the start in memory of the reserved portion of size number_of_bytes. If memory cannot be allocated a NULL pointer is returned.

Since a void * is returned the C standard states that this pointer can be converted to any type. The size_t argument type is defined in stdlib.h and is an unsigned type.

So:

```
char *cp;
cp = malloc(100);
```

attempts to get 100 bytes and assigns the start address to cp.

Also it is usual to use the sizeof() function to specify the number of bytes:

```
int *ip;
ip = (int *) malloc(100*sizeof(int));
```

Some C compilers may require to cast the type of conversion. The (int *) means coercion to an integer pointer. Coercion to the correct pointer type is very important to ensure pointer arithmetic is performed correctly.

It is good practice to use sizeof() even if you know the actual size you want – it makes for device independent (portable) code.

sizeof can be used to find the size of any data type, variable or structure. Simply supply one of these as an argument to the function.

So:

inti;

```
struct COORD {float x,y,z};
typedef struct COORD PT;
sizeof(int), sizeof(i),
sizeof(struct COORD) and
sizeof(PT) are all ACCEPTABLE
```

In the above, we can use the link between pointers and arrays to treat the reserved memory like an array, i.e, we can do things like:

```
ip[0] = 100;
```

or

```
for(i=0;i<100;++i) scanf("%d",ip++);
```

When you have finished using a portion of memory you should always free() it. This allows the memory freed to be available again, possibly for further malloc() calls.

The function free() takes a pointer as an argument and frees the memory to which the pointer refers.



Example:

```
#include <stdlib.h>

int main(){
    int *ptr;
    ptr = malloc(10 * sizeof(*ptr));
    if (ptr != NULL) {
        *(ptr + 7) = 80;

        printf("Value of the 8th integer is %d\n",*(ptr + 7));
        printf("Address of 8th integer is %d\n",(ptr+7));
    }
    return 0;
}
```



Example: #include <stdio.h>

```
#include <stdlib.h>
```

```
int main()
{

    int* ptr;
    int n, i;
```

```
// Get the number of elements for the array
n = 5;
printf("Enter number of elements: %d\n", n);

// Dynamically allocate memory using malloc()
ptr = (int*)malloc(n * sizeof(int));

// Check if the memory has been successfully
// allocated by malloc or not
if (ptr == NULL) {
    printf("Memory not allocated.\n");
    exit(0);
}
else {

    // Memory has been successfully allocated
    printf("Memory successfully allocated using malloc.\n");

    // Get the elements of the array
    for (i = 0; i < n; ++i) {
        ptr[i] = i + 1;
    }

    // Print the elements of the array
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);
    }
}

return 0;
}
```

Output:

```
Enter number of elements: 5
Memory successfully allocated using malloc.
The elements of the array are: 1, 2, 3, 4, 5,
```

12.3 calloc and realloc**calloc**

It is used to dynamically allocate the specified number of blocks of memory of the specified type. The `malloc()` function allocates memory and leaves the memory uninitialized. but, the `calloc()` function allocates memory and initializes all bits to zero. It returns `NULL` if memory is not sufficient.

realloc

If already allocated dynamically memory is insufficient or more than required, in program we can change the size of previously allocated memory using the `realloc()` function.

It is used to dynamically change the memory allocation of a previously allocated memory. re-allocation of memory maintains the already present value and new blocks will be initialized with default garbage value. re-allocation of memory maintains the already present value and new blocks will be initialized with default garbage value.

There are two allocation functions, `calloc()` and `realloc()`. Their prototypes are given below:

```
void *calloc(size_t num_elements, size_t element_size);
```

```
void *realloc(void *ptr, size_t new_size);
```

`malloc` does not initialise memory (to zero) in any way. If you wish to initialise memory then use `calloc`. `calloc` there is slightly more computationally expensive but, occasionally, more convenient than `malloc`. Also note the different syntax between `calloc` and `malloc` in that `calloc` takes the number of desired elements, `num_elements`, and `element_size`, `element_size`, as two individual arguments.

Thus to assign 100 integer elements that are all initially zero you would do:

```
int *ip;
ip = (int *) calloc(100, sizeof(int));
```

`realloc` is a function which attempts to change the size of a previous allocated block of memory. The new size can be larger or smaller. If the block is made larger then the old contents remain unchanged and memory is added to the end of the block. If the size is made smaller then the remaining contents are unchanged.

If the original block size cannot be resized then `realloc` will attempt to assign a new block of memory and will copy the old block contents. Note a new pointer (of different value) will consequently be returned. You must use this new value. If new memory cannot be reallocated then `realloc` returns `NULL`.

Thus to change the size of memory allocated to the `*ip` pointer above to an array block of 50 integers instead of 100, simply do:

```
ip = (int *) realloc(ip, 50);
```

C language requires the number of elements in an array to be specified at compile time. But it is not practically possible with arrays. In arrays we allocate the memory first and then start using it. This may result in failure of a program or wastage of memory space.

The concept of dynamic memory location can be used to eradicate this problem. In this technique, the allocation of memory is done at run time. C language provides four library functions known as memory management functions that can be used for allocating and freeing memory during program execution. These functions help us to build complex application programs that use the available memory intelligently.



Example: Calloc

```
#include <stdio.h>
int main() {
    int * ptr;
    ptr = calloc(10, sizeof(int));
```

```

if (ptr == NULL) {
    printf("Error! memory not allocated.");
    exit(0);
}
printf("Dynamically Building 10 Blocks ");
return 0;
}

```

**Example: realloc**

```

#include<stdio.h>
int main()
{
    char *ptr;
    ptr = NULL;
    ptr = realloc(ptr,10);
    if(ptr != NULL)
        printf("Memory created successfully\n");
    return 0;
}

```

12.4 free

The memory allocated using functions malloc() and calloc() is not de-allocated on their own. It is used to dynamically de-allocate the memory.

Syntax: free(ptr);

**Example: free**

```

#include <stdio.h>
int main() {
    int* ptr = malloc(10 * sizeof(*ptr));
    if (ptr != NULL){
        *(ptr + 4) = 50;
        printf("Value of the 4th integer is %d\n",*(ptr + 4));
    }
    free(ptr);
    return 0;
}

```

**Example**

```

#include <stdio.h>

```

```
#include <stdlib.h>

int main()
{
    int *ptr, *ptr1;
    int n, i;

    // Get the number of elements for the array
    n = 5;
    printf("Enter number of elements: %d\n", n);

    // Dynamically allocate memory using malloc()
    ptr = (int*)malloc(n * sizeof(int));

    // Dynamically allocate memory using calloc()
    ptr1 = (int*)calloc(n, sizeof(int));

    // Check if the memory has been successfully
    // allocated by malloc or not
    if (ptr == NULL || ptr1 == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else {

        // Memory has been successfully allocated
        printf("Memory successfully allocated using malloc.\n");

        // Free the memory
        free(ptr);
        printf("Malloc Memory successfully freed.\n");

        // Memory has been successfully allocated
        printf("\nMemory successfully allocated using calloc.\n");

        // Free the memory
        free(ptr1);
        printf("Calloc Memory successfully freed.\n");
    }

    return 0;
}
```


}

Output:

Enter number of elements: 5

Memory successfully allocated using malloc.

Malloc Memory successfully freed.

Memory successfully allocated using calloc.

Calloc Memory successfully freed.

Dynamic Memory Management Functions

Function	Typical call	Description
malloc	malloc (sz)	Allocate a block of size <i>sz</i> bytes from memory heap and return a pointer to the allocated block e.g., ptr = (cast.type*) malloc (byte_size);
calloc	calloc in (sz)	Allocate a block of size <i>n</i> x <i>sz</i> bytes from memory heap, initialize it to zero and return a pointer to the allocated block e.g., ptr = (cast_type*) calloc (n, elem_size);
realloc	realloc (blk,; sz)	Adjust the size of the memory block <i>blk</i> allocated on the heap to <i>sz</i> , copy the contents to a new location if necessary and return a pointer to the allocated block e.g., ptr = realloc (ptr, newsize);
free	free (blk)	Free block of memory <i>blk</i> allocated from memory heap e.g., free (ptr);

Advantages of Dynamic Memory allocation

- When we don't know how much memory will be required for the software ahead of time.
- When we need data structures that don't have a memory restriction.
- When you wish to make better use of your memory space.
- For example, if you allocate memory space for a 1D array like array[20] and only use 10 memory spaces, the remaining 10 memory spaces will be squandered, and this wasted memory will be unavailable to other programme variables.
- Insertions and deletions in dynamically constructed lists may be done quickly and easily by manipulating addresses, whereas insertions and deletions in statically allocated memory result in additional movements and memory waste.
- Dynamic memory allocation is required when using the concepts of structures and linked lists in programming.

Summary

Dynamic memory allocation is to allocate memory at “run time”.

Dynamically allocated memory must be referred to by pointers.

Dynamic Memory Allocation can be defined as a procedure in which the size of a data structure (like Array) is changed during the runtime.

Malloc method in C is used to dynamically allocate a single large block of memory with the specified size.

free method in C is used to dynamically de-allocate the memory.

realloc method in C is used to dynamically change the memory allocation of a previously allocated memory.

Keywords

Dynamic memory allocation: DMA perform manual memory allocation.

free: The free() function used to release space.

malloc: malloc() function reserves a block of memory of the specified number of bytes.

calloc: The calloc() function allocates memory and initializes all bits to zero.

Self Assessment

- Among 4 header files, which should be included to use the memory allocation functions like malloc(), calloc(), realloc() and free()?
 - #include<string.h>
 - #include<stdlib.h>
 - #include<memory.h>
 - All of above
- Which of the following statement is correct prototype of the malloc() function in c ?
 - int* malloc(int);
 - Char* malloc(char);
 - unsigned int* malloc(unsigned int);
 - void* malloc(size_t);
- malloc() returns a float pointer if memory is allocated for storing float's and a double pointer if memory is allocated for storing double's. A.
 - True
 - False
 - May Be
 - Can't Say
- DMA stands for
 - Dynamite Memory Access
 - Dynamic Memory Available
 - Direct Memory Access
 - None of Above
- Which of the following is memory allocation technique
 - Static Memory Allocation
 - Dynamic Memory Allocation
 - All of above
 - None of above

6. There is no memory reusable in
 - A. Static Memory Allocation
 - B. Dynamic Memory Allocation
 - C. All of above
 - D. None of above
7. Dynamic memory allocation is
 - A. More efficient
 - B. Less efficient
 - C. Don't know
 - D. None of above
8. Which function is used to delete the allocated memory space?
 - A. Dealloc()
 - B. free()
 - C. Both A and B
 - D. None of the above
9. Which of the following is/are true
 - A. calloc() allocates the memory and also initializes the allocated memory to zero, while memory allocated using malloc() has random data.
 - B. malloc() and memset() can be used to get the same effect as calloc()
 - C. Both malloc() and calloc() return 'void *' pointer
 - D. All of the above
10. Specify the 2 library functions to dynamically allocate memory?
 - A. malloc() and memalloc()
 - B. alloc() and memalloc()
 - C. malloc() and calloc()
 - D. memalloc() and faralloc()
11. calloc() returns a storage that is initialized to.
 - A. Zero
 - B. Null
 - C. Nothing
 - D. One
12. Which one is used during memory deallocation in C?
 - A. remove(p);
 - B. delete(p);
 - C. free(p);
 - D. terminate(p);

13. In which technique memory allocated at run time

- A. Static Memory Allocation
- B. Dynamic Memory Allocation
- C. All of above
- D. None of above

14. Syntax of calloc() is

- A. ptr = (castType*)calloc(n, size);
- B. ptr =calloc(n, size);
- C. ptr = (castType*) (n, size);
- D. ptr = (castType*)calloc();

15. In this program the allocated memory block can store

```
#include<stdio.h>

#include<stdlib.h>

int main()
{
int *ptr;
ptr = malloc(10);
return 0;
}
```

- A. int
- B. char
- C. float
- D. all of above

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. B | 2. D | 3. B | 4. C | 5. C |
| 6. A | 7. A | 8. B | 9. D | 10. C |
| 11. A | 12. A | 13. A | 14. A | 15. D |

Review Questions

1. What are the disadvantages of static memory allocation?
2. Differentiate between static memory allocation and dynamic memory allocation.
3. Write a program to demonstrate the malloc() function.
4. Differentia between free and realloc() function.
5. What are the advantages of dynamic memory allocation?



Further Readings

Books Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson

Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming with C", Tata McGraw Hill Publishing Company Limited, New Delhi

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

YashvantKanetkar, Let us C

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.



Web Links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com

www.geeksforgeeks.org

Unit 13: Structure and Union

CONTENTS

Objectives

Introduction

13.1 Structure Definition

13.2 Structure Initialization

13.3 Comparison of Structure Variables

13.4 Arrays of Structures

13.5 Array within Structures

13.6 Structures within Structures / nested structures

13.7 Passing Structures to Functions

13.8 Structure Pointers

13.9 Union – Definition and Declaration

13.10 Accessing a Union Member

13.11 Union of Structures

13.12 Initialization of a Union Variable

13.13 Uses of Union

13.14 Differences between Union and Structure

Summary

Keywords

Self-Assessment

Answer for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Define structure
- Identify array within structure
- Define union and deceleration
- Explain union of structures
- Differentiate between union and structure

Introduction

Arrays can be used to aggregate groups of variables of the same type, as you've seen. The difficulty now is how to combine data that isn't typed in the same way. The explanation is that C is a language that can be easily extended. It can be extended by defining data types that are built from the basic types. That example, you can use a data type called a structure to organize variables of various types.

The following topics are concerned in this unit:

1. Declaring and defining structures
2. Assigning values to structure members

3. Array within structures
4. Structure within structures
5. Passing structures to functions

13.1 Structure Definition

A structure is a collection of variables referenced under one name providing a convenient means of keeping related information together. The structure definition creates a format that may be used to declare structure variables in a program later on.

The general format of structure definition is as follows:

```
structtag_name
{
    data_type member1;
    data_type member2;
    -----
    -----
};
```

A keyword `struct` declares a structure to hold the details of fields of different datatypes. At this time, no variable has actually been created. Only a format of a new data type has been defined.

Consider the following example:

```
structaddr
{
    char name [30];
    char street [20];
    char city [15];
    char state [15];
    intpincode;
};
```

The keyword `struct` declares a structure to hold the details of fine fields of address, namely, `#name`, `street`, `city`, `state`, `pin code`. The first four members are character array and fifth one is an integer.

Creating Structure Variables

The structure declaration does not actually create variables. Instead, it defines data type only. For actual use a structure variable needs to be created. This can be done in two ways:

1. Declaration using tagname anywhere in the program.



Example: `struct book`

```
{
    char name [30];
    char author [25];
    float price;
}
struct book book1, book2;
```

2. It is also allowed to combine structure declaration and variable declaration in one statement.

This declaration is given below:

```

struct person
{
    char * name;
    int age;
    char *address;
}

p1, p2, p3;

```

While declaring structure variables along with their definition, the use of tag name is optional.

```

struct
{
    char *name;
    int age;
    char *address;
}

p1, p2, p3;

```

Giving Values to Members

As the members are not themselves variables they should be linked to the structure variables. The Link between a member and a variable is established using member operator '.' which is also known as dot operator.

This can be explained using following example:



Example: /* Program to define a structure and assign value to members*/

```

struct book
{
    char * name;
    int pages;
    char *author;
};

main()
{
    struct book b1;
    printf ("\n Enter Values:");
    scanf ("%s %d %s", b1.name, &b1.page, b1.author);
    printf ("%s, %d, %s, b1.name, b1.page, b1.author);
}

```

13.2 Structure Initialization

A structure variable can be initialized as any other data type.

```

main()
{
    static struct
    {
        int weight;
    }
}

```



```

float height;

}

student = {60, 180.75};

```

This assigns the value 60 to student.weight and 180.75 student.height. There is a one-to-one correspondence between the members and their initializing values.

A structure must be declared as static if it is to be initialized inside a function (similar to arrays). The following statements initialize two structure variables. Here, it is essential to use a tag

```

name.
main()
{
    structst_record
    {
        int weight;
        float height;
    }
    staticstructst_record student1 = {60, 180.75};
    staticstructst_record student2 = {53, 170.60};
    -----
    -----
}

```

Another method is to initialize a structure variable outside the function as shown below:

```

    structst_record / * No static word */
    {
        int weight;
        int height;
    }
    student1 = {60, 180.75};
}
main()
{
    staticstructst_record student 2 = {53, 170.60}
    -----
    -----
}

```

The initialization of individual structure members within the template is permitted. The initialization must be done only in the declaration of the actual variables.

13.3 Comparison of Structure Variables

Two variables of the same structure type can be compared the same way as ordinary variables.

If person1 and person2 belong to the same structure, then the following operations are valid.

Operation Meaning

person1 = person2 Assign person2 to person1.

person1 == person2 Compare all members of person1 and person2 and return 1 if they are equal, 0 otherwise.



Example:

```
struct class
{
    int number;
    char name [20];
    float marks;
}
main()
{
    int x;
    static struct class student1 = {111, "Rao", 72.50};
    static struct class student2 = {222, "Reddy", 67.00};
    static struct class student3;
    student3 = student2;
    x = ((student3.number == student1.number) && (student3.marks ==
    student2.marks)) ? 1:0;
    if (x == 1)
    {
        printf ("\nStudent2 and Student3 are same \n");
        printf ("%d %s %f\n", student3.number, student3.name,
        student3.marks);
    }
    else
    printf ("\nStudent2 and Student3 are different\n");
}
Output: Student2 and Student3 are same.
222 Reddy 67.000000
```

13.4 Arrays of Structures

The most common use of structures is in arrays of structures. To declare an array of structures, first the structure is defined then an array variable of that structure is declared. In such a declaration, each element of the array represents a structure variable.

Example: struct class student [100];

It defines an array called student which consists of 100 elements of structure named class.

An array of structures is stored inside the memory in the same way as a multi-dimensional array.



Example:

```

/* Program to implement an array of structure */

struct marks
{
    int sub1;
    int sub2;
    int sub3;
    int total;
};

main()
{
    inti;
    staticstruct marks student [3] = {{45, 67, 81, 0}, {75,
    53, 69, 0},
    {75, 53, 69, 0}, {57, 36, 71, 0}};
    staticstruct marks total;
    for (i = 0; i<= 2; i++)
    {
        student [i].total = student [i].sub1 + student [i].sub2+student[i] sub3;
        total.sub1 = total.sub1 + student [i].sub1;
        total.sub2 += student [i].sub2;
        total.sub3 += student [i].sub3;
        total.total = total.total + student [i].total;
    }
    printf ("STUDENT \t\t TOTAL \n");
    for (i = 0; i<= 2; i++)
    printf ("Student ]%d] \t \t %d \n", i+1, student [i].total);
    printf ("\n SUBJECT \t\t %d \n %s \t\t %d \n %s\t\t %d",
    "Subject1",
    total.sub1,
    "Subject2", total.sub2. "Subject3", total.sub3);
    printf ("\n GRAND TOTAL = \t\t %d \n", total.total.);
}

```

13.5 Array within Structures

Single or multi-dimensional arrays of type int or float can be defined as structure members.

Example:

```

struct marks
{
    int number;
    float subject [3];
}

student [2];

```

Here the member subject contains three elements, subject [0], subject [1], and subject [2]. These elements can be accessed using appropriate subscripts. For instance, the name student [1].subject [2]; would refer to the marks obtained in the third subject by the second student.

Example:

```
/* Program to implement arrays within a structure. */
main()
{
    struct marks
    {
        int sub [3];
        int total;
    };
    static struct marks student [3] = {45, 76, 81, 0, 75, 53, 69, 0, 57, 36,
    71,
    0};
    static struct marks total;
    int i, j;
    for (i = 0; i <= 2; i++)
    {
        student [i].total += student [i].sub[j]
        total.sub[j] += student [i].sub [j];
    }
    total.total += student [i].sub[j];
}
total.total += student [i].total;
}
printf ("STUDENT\t\tTOTAL \n");
}
for (i = 0; i <= 2; i++)
printf ("Student [%d]\t\t%d\n", i + 1, student [i].total);
printf("SUBJECT\t\t\t\t\t\n")
for (j = 0; j <= 2; j++)
printf ("Subject %d\t\t\t\t\t%d\n", j+1, total.sub[j]);
printf("GRAND TOTAL = \t\t\t\t\t%d\n", total.total);
}
```

Output: Student Total

Student[1] 193

Student[2] 197

Student[3] 164

Subject Total

Subject1 177

Subject2 156

Subject3 221

GRAND TOTAL 554

13.6 Structures within Structures / nested structures

Structures within a structure means nesting of structures. Let us consider the following structure defined to store information about the salary of employees.

```

struct salary
{
    char name [20];
    char department [10];
    int basic_pay;
    int dearness_allowance;
    int house_rent_allowance;
    int city_allowance;
}
employee;

```

This structure defines name, department, basic pay and three kinds of allowances. All the items related to allowance can be grouped together and declared under a sub-structure as shown below:

```

struct salary
{
    char name [20];
    char department [10];
    struct
    {
        int dearness;
        int house_rent;
        int city;
    }
    allowance;
}
employee;

```

The salary structure contains a member named allowance which itself is a structure with three members. The members contained in the inner structure, namely, dearness, house_rent and city can be referred to as:

```

employee.allowance.dearness
employee.allowance.house_rent
employee.allowance.city

```

Then inner most member in a nested structure can be accessed by chaining all the concerned structure variables (from outermost to inner most) with the member using dot operator.

The following statements are invalid:

```

employee.allowance      (actual member is missing)
employee.house_rent      (inner structure variable is missing)

```

An inner structure can have more than one variable. The following form of declaration is legal:

```

struct salary
{
    struct
    {
        int dearness;
        -----
    }
    allowance, arrears;
}

employee [100];

```

The inner structure has two variables, allowance and arrears. This implies that both of them have the same structure template.

A base member can be accessed as follows:

employee[1].allowance.dearness

employee[1].arrears.dearness

Tag names can also be used to define inner structures.



Example:

```

struct pay
{
    int dearness;
    inthouse_rent;
    int city;
};

struct salary
{
    char name [20];
    char department [10];
    struct pay allowance;
    struct pay arrears;
}

struct salary employee [100];

```

The pay template is defined outside the salary template and is used to define the structure of allowance and arrears inside the salary structure.

It is also permissible to nest more than one type of structures:

```

struct personal_record
{
    struct name_part name;
    struct date date_of_birth;
    -----
    -----
};

```

```
struct personal_record person1;
```

The first member of the structure is name which is of the type structname_part. Similarly, other members have their structure types.

13.7 Passing Structures to Functions

The values of a structure can be passed from one function to another using one of three approaches.

The first way is to pass each element of the structure as a function call's actual argument. The actual arguments are then processed as ordinary variables on their own.

The second way entails providing the full structure to the called function as a copy. Changes to the original structure are not reflected in the called function since the function works on a copy of the complete structure (in the calling function). As a result, it is required that the function return the full structure to the calling function.

The third approach employs a concept called pointers to pass the structure as an argument. In this case, the address location of the structure is passed to the called function. The function can access indirectly the entire structure and work on it.

The general format of sending a copy of a structure to the called function is:

```
function_name (structure_variable_name)
```

The called function takes the following form:

```
data_type function_name (st_name)
```

```
struct_type st_name;
```

```
{
```

```
-----
```

```
-----
```

```
return (expression);
```

```
}
```



Notes

1. The called function must be declared for its type, appropriate to the data type it is expected to return. For example, if it is returning a copy of the entire structure, then it must be declared as struct with an appropriate tag name.
2. The structure variable used as the actual argument and the corresponding formal argument in the called function must be of the same struct type.
3. The return statement is necessary only when the function is returning some data. The expression may be any simple variable or structure or an expression using simple variables.
4. When a function returns a structure, it must be assigned to a structure of identical type in the calling function.
5. The called function must be declared in calling function for its type, if it is placed after the calling function.



Lab Exercise

/* Program showing passing of structure member as function parameters */

```

struct stores
{
    char name [20];
    float price;
    int quantity;
};

main()
{
    struct stores update();
    float mul(), p_increment, value;
    int q_increment;
    static struct stores item = {"XYZ", 25.75, 12};
    printf("\nInput Increment Values:");
    printf("\nPrice Increment and Quantity Increment\n");
    scanf("%f %d", &p_increment, &q_increment);
    item = update(item, p_increment, q_increment);
    printf("\nUpdated values of item");
    printf("\nName : %s\n", item.name);
    printf("\nPrice : %f\n", item.price);
    printf("\nQuantity : %d\n", item.quantity);
    value = mul(item);
    printf("\nValue of the item: %d\n", value);
}

struct stores update(struct stores product, float p, int q)
{
    product.price += p;
    product.quantity += q;
    return (product);
}

float mul(struct stores stock)
{
    return (stock.price * stock.quantity);
}

```

Output: Input Increment Values: Price Increment and Quantity Increment

10 12

Updated values of item

Name : XYZ

Price : 35.750000

Quantity : 24

Value of the item : 858.000000

In case of structures having numerous structure elements passing these individual elements would be tedious. In such cases an entire structure can be passed to a function.

**Lab Exercise**

```

/* Program passing entire structure as function parameter. */

structemp
{
    charempname [25];
    char company [25];
    intempno;
}

main( )
{
    staticstructemp emp1 = {"Prashant", "SOCEM", 101};
    display (emp1);
}

display (e)
structemp e;
{
    printf ("%s\n%s\n%d", emp.empname, emp.company,
    emp.empno);
}

Output: Prashant
SOCEM
101

```

13.8 Structure Pointers

A complete structure can be transferred to a function by passing a structure-type pointer as an argument. In principle, this is similar to the procedure used to transfer an array to a function.

However, we must use explicit pointer notation to represent a structure that is passed as an argument. A structure passed in this manner will be passed by reference rather than by value.

Hence, if any of the structure members are altered within the function, the alterations will be recognized outside the function.

**Example:**

```

#include <stdio.h>

typedefstruct
{
    char *name;
    intacct_no;
    characcttype;
    float balance;
}

record;

/* transfer a structure-type pointer to a function */

```

```

main()
{
    void adjust (record *pt); /* function declaration */
    static record customer = {"Smith", 3333, 'C', 33.33};
    printf ("%s %d %c %.2f\n", customer.name, customer.acct_no,
    customer.acct_type, customer.balance);
    adjust (&customer);
    printf ("%s %d %c %.2f\n", customer.name, customer.acct_no,
    customer.acct_type, customer.balance);
}

void adjust (record *pt)
{
    pt->name = "Jones";
    pt->acct_no = 9999;
    pt->acct_type = 'R';
    pt->balance = 99.99;
    return;
}

```

This program illustrates the transfer of a structure to a function by passing the structure's address (a pointer) to the function. In particular, customer is a static structure of type record, whose members are assigned an initial set of values. These initial values are displayed when the program begins to execute. The structure's address is then passed to the function adjust where different values are assigned to the member of the structure.

Within adjust, the formal argument declaration defines pt as a pointer to a structure of type record. Also, nothing is explicitly returned from adjust to main. Within main, the current values assigned to the members of customer are again displayed after adjust has been accessed. Thus, the program illustrates whether or not the changes made in adjust carry over to the calling portion of the program.

Executing the program results in the following output:

```

Smith 3333 C 33.33
Jones 9999 r 99.99

```

The value assigned to the members of customer within adjust are recognized within main. A pointer to a structure can be returned from a function to the calling portion of the program. This feature may be useful when several structures are passed to a function, but only one structure is returned.

As we define a pointer pointing to int, or a pointer pointing to a char, similarly, we can have a pointer pointing to a struct. Such pointers are known as 'structure pointers'. The program given below demonstrates the usage of structure pointer.

```

main()
{
    structemp
    {
        charempname [25];
        char company [25];
        intempno;
    };
}

```

```

static struct emp1 = {"Prashant", "SOCEM", 101};

struct emp *ptr;

ptr = &emp1;

printf ("%s %s %d\n", emp1.empname, emp1.company, emp1.empno);

printf ("%s %s %d\n", ptr->company, ptr->empno);

}

```

In the above program, two types of operators are used to refer to structure elements:

1. Dot Operator
2. Arrow Operator

When the structure is referred to by its name, the structure elements are addressed using dot

operators.

Example: b1.name

When the structure is referred to by the pointer to structure, the structure elements are addressed

using arrow operators.

Example: ptr->name

On the left hand side of '.' structure operator, there must always be a structure variable, whereas on the right hand side of the '->' operator there must always be a pointer to a structure.

The following program demonstrates the passing of address of a structure variable to a function.

```

struct emp
{
    char empname [25];
    int empno;
}

main()
{
    static struct emp1 = {"Prashant", "socem", 101};
    display (&emp1);
}

display (e)
struct emp *e; /*pointer to a structure */
{
    printf ("%s \n%s\n%d", e->empname, e->empno);
}

```

Output: Prashant

SOCEM

101

In the above example, -> operator is used to access the structure elements using pointer to structure.

13.9 Union – Definition and Declaration

Unions follow the same syntax as structures but differ in terms of storage. In structures, each member has its own storage location, whereas all the members of a union use the same location.

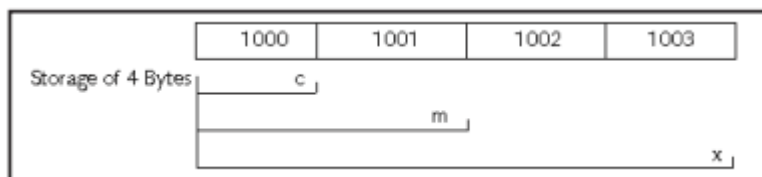
This implies that, although a union may contain many members of different types, it can handle only one member at a time.

Like structures, a union can be declared using the keyword `union` as follows:

union item

```
{
    int m;
    float x;
    char c;
} code;
```

This declaration declares a variable `code` of type `union item`. The union contains three members, each with a different data type. However, only one can be used at a time. This is due to the fact that only one location is allocated for a union variable, irrespective of its size.



The compiler allocates a piece of storage that is large enough to hold the largest variable type in the union. As shown in the example declaration, the member `x` requires 4 bytes which is the largest among the members. It is assumed that a float variable requires 4 bytes of storage and the figure above shows how all the three variables share the same address.

13.10 Accessing a Union Member

To access a union member, you can use the same syntax that you use for structure members.

Example: `code.m`, `code.x`, `code.c` are all valid member variables.

During accessing, you should make sure that you are accessing the member whose value is currently stored.

Example: The statements such as

```
code.m = 150;
code.x = 785;
printf ("%d", code.m);
```

would produce erroneous output (which is machine dependent). The user must keep track of what type of information is stored at any given time.

Thus, a union creates a storage location that can be used by any one of its members at a time. When a different member is assigned a new value, the new value supersedes the previous member's value.

Unions may be used in all places where a structure is allowed. The notation for accessing a union member which is nested inside a structure remains the same as for the nested structures.

13.11 Union of Structures

Just as one structure can be nested within another, a union too can be nested in another union. Not only that, there can be a union in a structure, or a structure in a union. Here is an example of structures nested in a union.

**Lab Exercise**

```

main( )
{
    struct a
    {
        inti;
        char c[2];
    };
    struct b
    {
        int j;
        char d[2];
    };
    union z
    {
        struct a key;
        struct b data;
    }strange;
    strange.key.i = 512;
    strange.data.d[0] = 0;
    strange.data.d[1] = 32;
    printf("%d\n", strange.key.i);
    printf("%d\n", strange.data.j);
    printf("%d\n", strange.key.c[0]);
    printf("%d\n", strange.data.d[0]);
    printf("%d\n", strange.key.c[1]);
    printf("%d\n", strange.data.d[1]);
}

```

Output: 512

512

0

0

32

32

Structures and unions may be freely mixed with arrays.

**Example:**

```

union id
{
    char color[12];
}

```

```

int size;

};

struct clothes
{
    char manufacturer[20];
    float cost;
    union id description;
} shirt, trouser;

```

Now shirt and trouser are structure variable of type clothes. Each variable will contain the following members: a string (manufacturer), a floating-point quantity (cost), and a union (description). The union may represent either a string (color), or an integer quantity (size). Another way to declare the structure variable shirt and trouser is to combine the above two declarations. This is shown as follows:

```

struct clothes
{
    char manufacturer[20];
    float cost;
    union {
        char color[12];
        int size;
    } description;
} shirt, trouser;

```

This declaration is more concise, though perhaps less straightforward than the original declarations.

An individual union member can be accessed in the same manner as an individual structure member, using the operators “.” and “->”. Thus, if variable is a union variable, then variable.member refers to a member of the union. Similarly, if ptvar is a pointer variable that points to a union, then ptvar->member refers to a member of that union.



Example:

```

#include <stdio.h>

main()
{
    union id
    {
        char color;
        int size;
    };
    struct
    {
        char manufacturer[20];
        float cost;
        union id description;
    } shirt, trouser;
    printf("%d\n", sizeof(union id));
}

```

```

shirt.description.color = ' w ' ; /* assigns a value to color */
printf("%c %d\n", shirt.description.color, shirt.description.size);
shirt.description.size = 12; /* assigns a value to size */
printf("%c %d\n", shirt.description.color, shirt.description.size);
}

```

13.12 Initialization of a Union Variable

A union variable can be initialized, provided its storage class is either external or static. Only one member of a union can be assigned a value at any one time. The initialization value is assigned to the first member within the union.



Example:

Program to demonstrate initialization of union variables.

```

#include <stdio.h>

main( )
{
    union id
    {
        char color[12];
        int size;
    };
    struct clothes
    {
        char manufacturer[20];
        float cost;
        union id description;
    };
    static struct clothes shirt = {"American", "25.00", "White"};

    printf("%d\n", sizeof(union id));
    printf("%s %5.2f", shirt.manufacturer, shirt.cost);
    printf("%s %d\n", shirt.description.color, shirt.description.size);
    shirt.description.size = 12;
    printf("%s %5.2f", shirt.manufacturer, shirt.cost);
    printf("%s %d\n", shirt.description.color, shirt.description.size);
}

```

Output: 12

American 25.00 White 26743

American 25.00 ~ 12

13.13 Uses of Union

Unions, like structures, contain members whose individual data types may differ from one another. But the members that compose a union share the same storage area within the computer's memory,

whereas each member within a structure is assigned its own unique storage area. Thus, unions are used to conserve memory.

Unions are useful for applications involving multiple members, where values need not be assigned to all of the members at any one time. Unions are also used wherever the requirement is to access the same memory locations in more than one way. This is often required while calling Basic Input/Output System functions (often simply called BIOS routines) present in the read only memory (ROM) of the computer.

Many DOS based application software's need to access DOS internal data structures. The breakup of these internal data structures however, is not consistent and often changes from one version of DOS to another. Therefore, to make the application programs compatible with different versions of DOS, these programs create unions which take into account the variations in the breakup of these DOS data structures. These programs when executed first test the version number of DOS being used on the machine and then access the appropriate part of the union.

13.14 Differences between Union and Structure

The differences between structure and union are:

1. Union allocates the memory equal to the maximum memory required by the member of the union but structure allocates the memory equal to the total memory required by the members.
2. In union, one block is used by all the member of the union but in case of structure, each member have their own memory space.
3. Union is best in the environment where memory is less as it shares the memory allocated. But structure cannot implemented in shared memory.
4. As memory is shared, ambiguity are more in union, but less in structure.
5. Self referential union cannot be implemented in any data structure ,but self referential structure can be implemented.

Summary

- Structure is a derived data type used to store the instances of variables of different data types.
- Structure definition creates a format that may be used to declare structure variables in the program later on.
- The structure operators like dot operator "." are used to assign values to structure members.
- Structure variable can be initialized as any other data type. An array of structure can be declared as any other array. In such an array, each element is a structure. Structures may contain arrays as well as structures.
- Union is a memory location that is shared by two or more variables.
- When union variable is declared, compiler automatically allocates enough storage to hold to largest member of union. Only the unions with storage class external or static can be initialized.
- Unions are useful for applications involving multiple members. They are also used in many
- DOS based application softwares. typedef and enum are two user defined data types.

Keywords

Random access file: A file, which allows accessing its records without restriction on the order of access.

Sequential file: A file, which allows accessing its records only from the first record onwards.

Structure: A grouped data type created by user.

Programming in C

Structure: A structure is a collection of variables referenced under one name providing a convenient means of keeping related information together. Structures within structure: It means nesting of structures.

Union: A data type that allows more than one variable to share the same memory area. User defined data types: The way you are not creating any new data type but are referring to an existing data type by a different name. Such data types are known as user defined data types.

Self-Assessment

1. A structure is a ____
 - A. collection of variables (different types) represented by single name
 - B. A structure is a user defined data type in C
 - C. Keyword 'struct' is used to define structure in C
 - D. All of above

2. Structure members are accessed using ____
 - A. :
 - B. .
 - C. >
 - D. <

3. Which keyword is used to define structure in C ____
 - A. structure
 - B. struct
 - C. structC
 - D. none of above

4. Passing structure in function using ____
 - A. function by value
 - B. function by reference
 - C. both function by reference and function by value
 - D. none of above

5. How to access members using Pointer ____
 - A. Using indirection (*) operator and dot (.) operator
 - B. Using arrow (->) operator or membership operator
 - C. Both using (.) (*) and (->)
 - D. none of above

6. Which of the following operation is illegal in structures?
 - A. Pointer to a variable of the same structure
 - B. Dynamic allocation of memory for structure
 - C. Typecasting of structure
 - D. All of the mentioned

7. Self-Referential Structures are _____

- A. Have one or more pointers which point to the same type of structure, as their member.
- B. Can dynamically be expanded or contracted
- C. Both of them
- D. None of above

8. What are the types of Self Referential Structures?

- A. Structure with Single Link
- B. Structure with Multiple Links
- C. Both Structure with Single Link and Structure with Multiple Links
- D. None of the above

9. Union is _____

- A. user define data type available in C
- B. only one member of the union can occupy the memory
- C. the size of the union in any instance is equal to the size of its largest element
- D. all of above

10. How to access union member?

- A. (->)
- B. (+)
- C. (.)
- D. (:)

11. Union differs from structure in the following way

- A. All members are used at a time
- B. Union cannot have more members
- C. Only one member can be used at a time
- D. Union initialized all members as structure

12. Which of the following cannot be a structure member?

- A. Array
- B. Structure
- C. Function
- D. None of the above

13. Which of the following is a collection of multiple data types in and of itself?

- A. String
- B. Structures
- C. Char
- D. None of the above

14. Point out the error in the program?

```
structemp  
{
```

Programming in C

```
intcode;  
structemp *e;  
};
```

- A. Error: in structure declaration
- B. Linker Error
- C. No Error
- D. None of above

15. Which of the following is correct about structures?

- A. Structure elements can be initialized at the time of declaration.
- B. Structure members can not be initialized at the time of declaration
- C. Only integer members of structure can be initialized at the time of declaration
- D. None of the above

Answer for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. B | 3. B | 4. C | 5. C |
| 6. C | 7. C | 8. C | 9. D | 10. A |
| 11. C | 12. C | 13. B | 14. C | 15. B |

Review Questions

1. What do you mean by 'Structure'? How it can be declared and initialized in a C program?
2. Draw a diagram to represent the internal storage of a structure.
3. What do you mean by 'Union'? How it can be declared and initialized in a C program?
4. Differentiate the followings:
 - (a) Arrays and structures
 - (b) Local and global structure
 - (c) Array of structure and structure within array
 - (d) Structure and union
5. Write short note on:
 - (a) Internal storage of union
 - (b) Function returning structures
 - (c) Structure within a structure
6. Write a program that compares two given dates. To store date use structure say date that contains three members namely date, month and year. If the dates are equal then display message as "Equal" otherwise "Unequal".
7. Explain the usefulness of structures and unions in C.

**Further Readings**

Books Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education,

Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

Yashvant Kanetkar, Let us C



Web links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com

Unit 14: Structures

CONTENTS

Objectives

Introduction

14.1 What is a File?

14.2 Defining and Opening a File

14.3 Input/output Operations on Files

14.4 Errors during Input/output

14.5 Types of Files: Text and Binary files

14.6 Reading, writing and append in file

14.7 Header files

Summary

Keywords

Self-Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Know how to define and opening a file
- Perform input/output operation on files
- Appending in files
- Preprocessor Directives and Macros

Introduction

Data is only stored in variables and arrays for a limited time. When a programme ends, all of this data is lost. Files are used to store enormous volumes of data indefinitely. Secondary storage devices, particularly disc storage devices, are used by computers to store files. We'll go through how C programmes edit, update, and process data files in this unit.

A file is a set of data or characters that can be text or a programme. In the C programming language, there are two sorts of files: sequential and random access. Random access files are more difficult to create than sequential ones. The data or text will be stored or read back sequentially. In random access file, the data can be accessed and processed randomly

14.1 What is a File?

Wherever there is a need to handle large volumes of data, it is advantageous to store data on the disks and read whenever necessary. This method employs the concept of files to store data. A file is a place on disk where a group of related data is stored. C supports a number of functions that have the ability to perform basic file operations, which include:

1. Naming a file
2. Opening a file

Programming in C

3. Reading data from a file
4. Writing data to a file
5. Closing a file

There are two distinct ways to perform file operations in C.

1. Low level I/O Operation (It uses operating system calls)
2. High level I/O Operation (It uses functions in C's Standard I/O library)

List of High Level I/O Functions

Function Name	Operation
<code>fopen()</code>	Creates a new file for use or opens an existing file for use.
<code>fclose()</code>	Closes a file which has been opened for use.
<code>getc()</code>	Reads a character from a file.
<code>putc()</code>	Writes a character to a file.
<code>fprintf()</code>	Writes a set of data values to a file.
<code>fscanf()</code>	Reads a set of data values from a file.
<code>getw()</code>	Reads an integer from a file.
<code>putw()</code>	Writes an integer to a file.

14.2 Defining and Opening a File

Before storing data in a file in the secondary memory, certain things about the file must be specified to the operating system. These include:

1. Filename
2. Data Structure
3. Purpose

Filename

It is a string of characters that makes up a valid filename for an operating system. It may contain two parts, a primary name and an optional period with an extension.

Example:

Input.dat

Store

PROG.C

Student.C

Text.out

Data Structure

Data structure of a file is defined as file in the library of standard I/O function definitions. All files should be declared as of type `FILE` before they are used.

Purpose: When we open a file, we must specify what we want to do with the file.

Following is the general format for declaring and opening a file:

```
FILE *fp;
```

```
fp = fopen ("filename", "mode");
```

The first statement declares the variable `fp` as a "pointer to the data type file".

The second statement opens the file named filename and assigns an identifier to the file type pointer fp. This pointer which contains all the information about the file is subsequently used as a communication link between the system and the program.

The second statement also specifies the purpose of opening this file. The mode does this job.

Mode can be one of the following:

- r Opens the file for reading only.
- w Opens the file for writing only.
- A Opens the file for appending (or adding) data to it.

Both the filename and mode are specified as string. They should be enclosed in double quotation marks.

Depending on the mode specified, one of the following actions may be performed:

1. When the mode is 'writing', a file with the specified name is created, if the file does not exist. The contents are deleted, if the file already exists.
2. When the purpose is 'appending', the file is opened with the current contents safe. A file with the specified name is created if the file does not exist.
3. If the purpose is 'reading', and if it exists, then the file is opened with the current contents safe, otherwise an error occurs.

Other additional modes of operation are:

File opening modes

Modes	Description
r	opens a text file in reading mode
w	opens or create a text file in writing mode.
a	opens a text file in append mode
r+	opens a text file in both reading and writing mode
w+	opens a text file in both reading and writing mode
a+	opens a text file in both reading and writing mode
rb	opens a binary file in reading mode

Whenever a file is opened using fopen function, a file pointer is returned. If the file cannot be opened for some reason, then the function returns a null pointer.

This facility can be used to test whether a file has been opened or not.



Example:

```
if (fp == NULL)
    printf ("File could not be opened.\n");
```

13.3 Closing a File

Programming in C

Once all the operations on a file have been completed, the file is closed. This is done to clear the buffers and flush all the information associated with the file. It also prevents any accidental misuse of the file. In case there is a limit to the number of files that can be kept open simultaneously, closing of unwanted files might help open the required files. When there is a need to use a file in a different mode, the file has to be first closed and then reopened in a different mode.

The I/O library supports a function for this of the following form:

```
fclose (file_pointer);
```

This would close the file associated with the file pointer `file_pointer`.

**Example:**

.....

```
FILE *p1, *p2;
p1 = fopen ("INPUT", "w");
p2 = fopen ("OUTPUT", "r");
.....
fclose(p1);
fclose(p2);
```

This program opens two files and closes them after all operations on them are completed.

Once a file is closed, its file pointer can be reused for another file. All files are closed automatically whenever a program terminates. However, closing a file as soon as all operations related to it have been completed is a good programming habit.

**Example:**

```
#include<stdio.h>
int main(){
FILE *fp;
fp=fopen(" abc.txt","r");
if(fp==NULL)
{
printf("Error, In opening file");
exit(1);
}
fclose(fp);
return 0;
}
```

14.3 Input/output Operations on Files**getc&putc Functions**

These are analogous to `getchar` and `putchar` functions and can handle only one character at a time.

`putc` can be used to write a character in a file opened in write mode.

A statement like `putc (ch, fp1);` writes the character contained in the character variable `ch` to the file associated with file pointer `fp1`.

Similarly, `getc` is used to read a character from a file that has been opened in read mode.

The statement `c = getc(fp2);` would read a character from the file whose file pointer is `fp2`.

The file pointer moves by one character position for every operation of `getc` or `putc`. The `getc` will return an end-of-file marker EOF, when end of the file has been reached. The reading should be terminated when EOF is encountered. Testing for the end-of-file condition is important. Any attempt to read past the end of file might either cause the program to terminate with an error or result in an infinite loop situation.

getw&putw Functions

The `getw` and `putw` are integer-oriented functions. They are similar to the `getc` and `putc` functions and are used to read and write integer values on unix systems.

The general forms of `getw` and `putw` are:

```
putw (integer, fp); &getw (fp);
```

fprintf & fscanf Functions

The functions `fprintf` and `fscanf` perform I/O operations that are identical to the familiar `printf` and `scanf` functions.

The general syntax of `fprintf` is

```
fprintf (fp, "control string", list);
```

where `fp` is a file pointer associated with a file that has been opened for writing. The control string contains output specifications for items in the list. The list may include variables, constants and strings.



Example: `fprintf (f1, "%s %d %f", name, age, 7.5);` here `name` is an array variable of type `char` and `age` in an `int` variable.

The general syntax of `fscanf` is

```
fscanf (fp, "control string", list);
```

This statement would cause the reading of the items in the list from the file specified by `fp`, according to the specifications contained in the control string.



Example: `fscanf (f2, "%s %d", item, &quantity);`

`fscanf` also returns the number of items that are successfully read. When the end of file is reached, it returns the value EOF.

feof() Function

The `feof` function can be used to test for an end of file condition. It takes a file pointer as its only argument and returns a non-zero integer value if all of the data from the specified file has been read, and returns zero otherwise. If `fp` is a pointer to the file that has just been opened for reading, then the statement

```
if (feof (fp))
```

```
printf ("End of data.\n");
```

would display the message "End of data." on reaching the end of file condition.

14.4 Errors during Input/output

It is possible that an error may occur during Input/Output operations on a file. Typical error situations include:

1. Trying to read beyond the end-of-file mark.
2. Device overflow.
3. Trying to use a file that has not been opened.
4. Trying to perform an operation on a file, when the file is opened for another type of operation.
5. Opening a file with an invalid filename.
6. Attempting to write to a write-protected file.

Programming in C

The `ferror` function reports the status of the file indicated. It also takes a file pointer as its argument and returns a non-zero integer if an error has been detected upto that point, during processing.

It returns zero otherwise.

The statement

```
if (ferror (fp) != 0)
```

```
printf ("An error has occurred. \n");
```

would print the error message, if the reading is not successful.

14.5 Types of Files: Text and Binary files

When dealing with files, there are two types of files you should know about:

Text files: - Text files are the normal .txt files. You can easily create text files using any simple text editors such as Notepad.

Binary files: -Binary files are mostly the .bin files in your computer.

Text file vs. binary file

Text File	Binary File
Bits represent character.	Bits represent a custom data.
Less prone to get corrupt as changes reflect as soon as the file is opened and can easily be undone.	Can easily get corrupted, even a single bit change may corrupt the file.
Can store only plain text in a file.	Can store different types of data (image, audio, text) in a single file.
Widely used file format and can be opened using any simple text editor.	Developed especially for an application and may not be understood by other applications.
Mostly .txt and .rtf are used as extensions to text files.	Can have any application defined extension.

14.6 Reading, writing and append in file**Reading in file**

In C Programming reading data block from a file is process of reading from file.

Syntax

```
fp=fopen("filename", "r");
```

```
while(fscanf(fp, "%s", str)!=EOF)
```

```
{
```

```
printf("%s",str);  
}
```

**Example:**

```
#include<stdio.h>  
  
int main()  
{  
    charstr[50];  
    FILE *fp;  
    fp=fopen(" abc.txt","r");  
    if(fp==NULL){  
        printf("-Error");  
        exit(1);  
    }  
    while(fscanf(fp, "%s", str)!=EOF)  
    {  
        printf("%s",str);  
    }  
    fclose(fp);  
    return 0;  
}
```

Writing in file

In C programming writing data block in file is possible.

Syntax

```
fp=fopen(" filename", "w");
```

**Example:**

```
#include<stdio.h>  
  
int main()  
{  
    char name[20]="this is C";  
    FILE *fp;  
    fp=fopen("abc.txt","w");  
    fprintf(fp, "%s",name);  
    printf("Contents written");  
    fclose(fp);  
    return 0;  
}
```

Append in file

In C Programming appends the content of file at the end of another is possible with append mode of file.

Syntax

```
fp=fopen("filename", "a");
```



Example:

```
#include<stdio.h>

int main()
{
    FILE *fp;
    fp=fopen("abc.txt","a");
    fprintf(fp, "\n of programming");
    printf("contents added in file");
    fclose(fp);
    return 0;
}
```

14.7 Header files

Header files provide predefined functions to make programming easier. header files contain the set of predefined standard library functions

A header file is a file with extension .h which contains C function declarations and macro definitions to be shared between several source files.

The C Preprocessor is not a part of the compiler, but is a separate step in the compilation process.

Before a C program is compiled in a compiler, source code is processed by a program called preprocessor. This process is called preprocessing.

Commands used in preprocessor are called preprocessor directives and they begin with “#” symbol.

Preprocessor

Preprocessor	Syntax/Description
Macro	Syntax: #define This macro defines constant value and can be any of the basic data types.
Header file inclusion	Syntax: #include <file_name> The source code of the file “file_name” is included in the main program at the specified place.
Conditional compilation	Syntax: #ifdef, #endif, #if, #else, #ifndef Set of commands are included or excluded in source program before compilation with respect to the condition.

Other directives	Syntax: #undef, #pragma #undef is used to undefine a defined macro variable. #Pragma is used to call a function before and after main function in a C program.
------------------	--

Header Files Types

Standard library

User-defined

Pre-existing header files: already available in C/C++ compiler user need to import them.

User-defined header files: defined by the user and can be imported using “#include”.

Standard Header Files:

#include<stdio.h>: functions scanf() and printf().

#include<math.h>: sqrt(), pow(), etc.

#include<iostream>: cin and cout.

#include<string.h>: strlen(), strcmp(), strcpy(), size()

#include<time.h> : date() and time()

Syntax of Header File

For Standard library: #include<filename>

For user defined: #include"filename"



Example:

For Standard library

```
#include<stdio.h>
```

```
#include<string.h>
```

For user defined

```
#include"myheader.h"
```



Example:

```
#include<stdio.h>
```

```
#include"header.h"
```

```
int main(){
```

```
printf("%d",fact(5));
```

```
return 0;
```

```
}
```

Macros

A macro is a segment of code which is replaced by the value of macro. Macro is defined by #define directive.

C Macros

Predefined

User-defined

**Example: Predefine**

```
#include<stdio.h>

int main(){
printf("File name : %s\n",__FILE__);
printf("File name : %s\n",__DATE__);
printf("File name : %s\n",__TIME__);
return 0;
}
```

**Example: User define**

```
#include<stdio.h>

#define pi 3.14

int main(){
int r=10,area;
area=pi*r*r;
printf("Area of circle is %d",area);
return 0;
}
```

Summary

- File is a collection of data or set of characters may be a text or program.
- There are two types of files used in the C language: sequential file and random access file.
- While creating a file, fopen() function is used which opens a stream for use and link it with program and file. fopen() function has two string arguments which represent the name of the file and the type of I/O to be performed. fclose() function closes a stream that was opened by a call to fopen(). putc() is used to transfer one character into file. getc() functions allows you to read data from a file.
- Similarly to read or write strings, fgets and fputs functions are used.
- The fgets() function reads a string from the file and copies it in a string variable lying in memory.
- fputs() is used to write a string in a data file.
- getw() function is used to read an integer from a file.
- putw() is used to write as integer value in a file. fprintf() and fscanf() are similar to printf() and scanf()
-

Keywords

Data Structure: Data structure of a file is defined as file in the library of standard I/O function definitions.

NULL: A system-defined value (not 0) that indicates various exceptional conditions such as end of a string, a pointer referencing nothing, etc.

Random access file: A file, which allows accessing its records without restriction on the order of access. You can access 60th record without accessing the 59th record, and so on.

Macros

User Define

Predefine

Self-Assessment

1. Text files are_____
 - A. Can store only plain text in a file.
 - B. Bits represent character
 - C. .txt and .rtf are used as extensions
 - D. All of above

2. What are the different file operations?
 - A. Creating a new file
 - B. Opening an existing file
 - C. Closing a file
 - D. Above all

3. Putc() is used for_____
 - A. reads a integer from a file
 - B. set the position to desire point
 - C. writes a character to a file
 - D. none of above

4. fseek() is used for_____
 - A. set the position to desire point
 - B. reads a integer from a file
 - C. writes a character to a file
 - D. none of above

5. r+ mode is used for
 - A. opens a binary file in reading mode
 - B. opens a text file in both reading and writing mode
 - C. opens or create a text file in writing mode.
 - D. All of above

6. EOF indicates _____
 - A. function closes the file and returns zero on success.
 - B. if there is an error in closing the file
 - C. gives current position in the file
 - D. none of above

7. Find incorrect statement
 - A. fp=fopen(" abc*txt","a");
 - B. fprintf(fp, "\n of programming");
 - C. printf("contents added in file");

- D. `fclose(fp);`
8. Which one is used for append mode_____
- A. `r`
 - B. `a`
 - C. `rb`
 - D. none of above
9. Header files are _____
- A. A header file is a file with extension `.h`
 - B. Header files provide predefined functions to make programming easier
 - C. header files contain the set of predefined standard library functions
 - D. all of above
10. What are the part of Conditional Compilation?
- A. `#endif`
 - B. `#define`
 - C. `#undef`
 - D. None of above
11. Extension of text file is
- A. `.com`
 - B. `.in`
 - C. `.txt`
 - D. `.ppt`
12. Macro is defined by_____
- A. `#endif`
 - B. `#define`
 - C. `#else`
 - D. `#ifdef`
13. How to import User-defined header files_____
- A. using `"#include"`.
 - B. using `"#ifdef"`.
 - C. using `"#define"`.
 - D. None of above
14. File type can be represented by_____
- A. File extension
 - B. File name
 - C. file identifier
 - D. none of above

15. A file is a sequence of _____

- A. bytes
- B. lines
- C. bits
- D. All of above

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. D | 3. C | 4. A | 5. B |
| 6. B | 7. A | 8. B | 9. D | 10. A |
| 11. C | 12. B | 13. A | 14. A | 15. D |

Review Questions

1. What do you mean by the “File Handling”? Explain the concept of file handling in C.
2. Explain the various type of files and their access mechanisms.
3. Write in detail about any five file-handling functions in C.
4. What do mean by random file access? How C implements the concept of random file access?
5. Write a program to read a file and display contents with its line numbers.
6. What is significance of Macros in programming?
7. Differentiate between user define and predefine Macros.



Further Readings

Books Ashok N. Kamthane, “Programming with ANCI & Turbo C”, Pearson Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, “The Programming Language”, Prentice Hall of India, New Delhi

Byron Gottfried, “Programming with C”, Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

YashvantKanetkar, Let us C



Web links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com